

SYSTEM DESIGN

PHASE 1 — SYSTEM DESIGN FUNDAMENTALS

📄 High-End Interview Notes • Simple English • Diagrams • Examples • Trade-offs

System Design = Turning Requirements into an Architecture

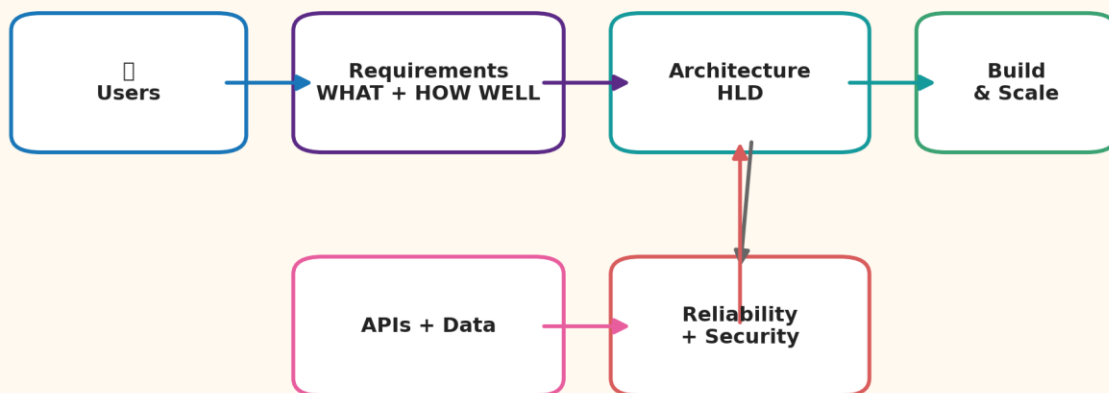


Figure 1 — The overall System Design thinking flow

🎯 What you should be able to do after this chapter

Given a system-design question, identify requirements, separate functional and non-functional needs, sketch an HLD, understand where LLD fits, define clear component boundaries, design basic APIs, and explain trade-offs.

1. What is System Design?

System Design is the process of deciding how different parts of a software system should work together to satisfy a set of requirements.

In simple English: you are deciding what components you need, how they communicate, where data lives, and how the system should behave when users and traffic increase.

ELI10

Think about building a city. You do not start by deciding the color of one house. First you decide roads, hospitals, schools, power, water, traffic flow, and where everything connects. System Design works in a similar way for software.

The 4 questions to keep asking

1. What does the system need to do?
2. How many users / requests / data do we need to support?
3. What components should exist and how should they communicate?
4. What happens when something becomes slow, overloaded, or fails?

Example — Food Delivery App: a complete design may involve a mobile/web client, API layer, authentication, restaurant service, order service, payment integration, database, cache, notification system, and monitoring.

Functional vs Non-Functional Requirements

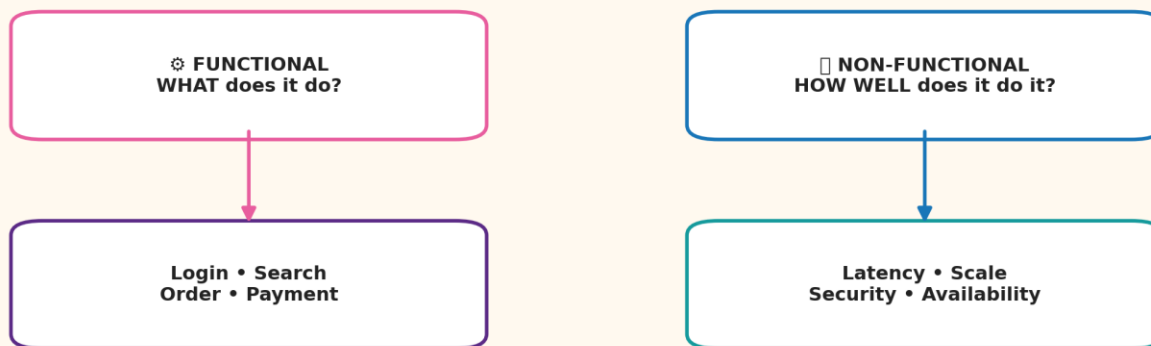


Figure 2 — Requirements are the starting point, not technology choices.

2. Functional Requirements

Functional requirements describe the features and actions the system must provide.

One-line memory

Functional = WHAT the system does.

Example — E-commerce App

- User can create an account and log in.
- User can search for products.
- User can view product details.
- User can add products to a cart.
- User can place an order.
- User can make a payment.
- User can track an order.

Functional requirement format

A useful way to write one is: “The system shall allow **actor** to **perform an action.”**

Actor	Action	Example
User	Search	User can search for products
User	Create	User can create an order
Admin	Manage	Admin can disable a product
System	Notify	System sends an order-status notification

Interview tip

Do not list 30 tiny features. Identify the core features needed to define the architecture. For a chat system, sending messages, receiving messages, delivery status, and user identity are usually more important than every UI detail.

3. Non-Functional Requirements

Non-functional requirements describe the quality, performance, reliability, and operational behavior expected from the system.

One-line memory

Non-functional = HOW WELL the system does its job.

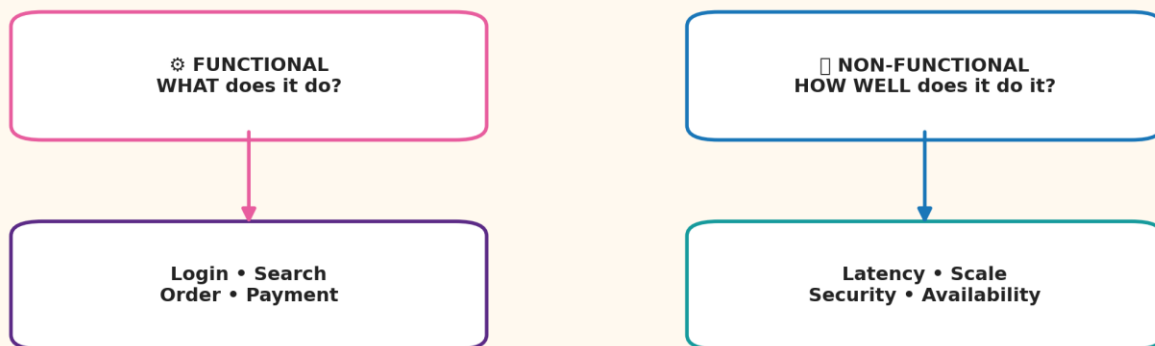
Common NFRs

Requirement	Simple meaning	Example
Performance	How fast?	API response under an agreed latency target
Scalability	Can it handle growth?	Support increasing traffic by adding capacity
Availability	Is it available?	Service should remain available despite some failures
Reliability	Does it work correctly?	Avoid duplicate payments
Security	Can it resist unauthorized access?	Protect user credentials and private data
Durability	Does saved data survive?	Successful orders should not disappear
Maintainability	Can engineers change it safely?	New features should not require rewriting everything

Functional + Non-functional together

Example: “Users can upload videos” is functional. “The system should support large uploads and remain responsive during peak traffic” is non-functional.

Functional vs Non-Functional Requirements





4. High-Level Design (HLD)

HLD is the big-picture architecture of the system. You focus on major components and how they communicate, not individual methods or lines of code.



HLD asks

“What components do we need, where do they run, what data do they own, and how do they communicate?”

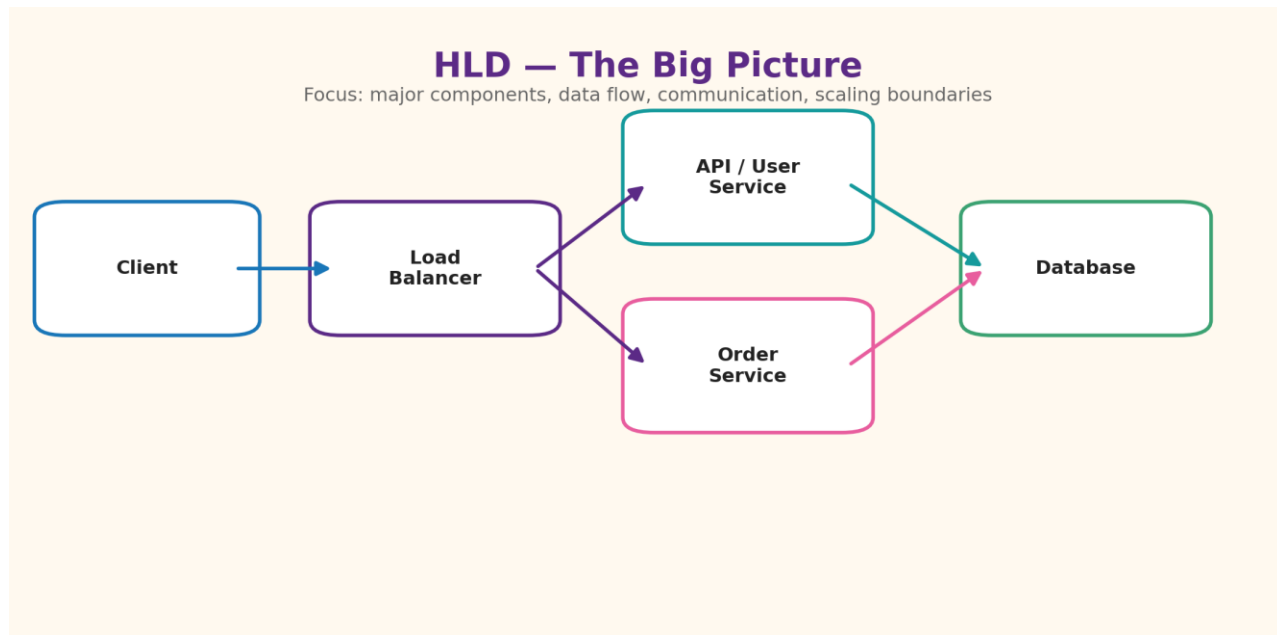


Figure 3 — HLD focuses on major building blocks and data flow.



What belongs in HLD?

- Clients / frontends
- Load balancers and reverse proxies
- Application services
- Databases
- Caches
- Message queues / brokers
- Object storage
- CDNs
- External services
- Major data flows



HLD example — YouTube

At HLD level, you might identify User Service, Video Service, Search Service, Object Storage, Database, CDN, Cache, and a background processing pipeline. You do not yet need to write the `VideoService.uploadVideo()` method.

HLD interview mistake to avoid

Do not immediately draw ten microservices because “big companies use microservices.” Start from requirements and choose the simplest architecture that satisfies them.

🔑 5. Low-Level Design (LLD)

LLD zooms inside one component and explains how it is implemented. It is much closer to code.

🧠 LLD asks

“How exactly should this component work internally?”

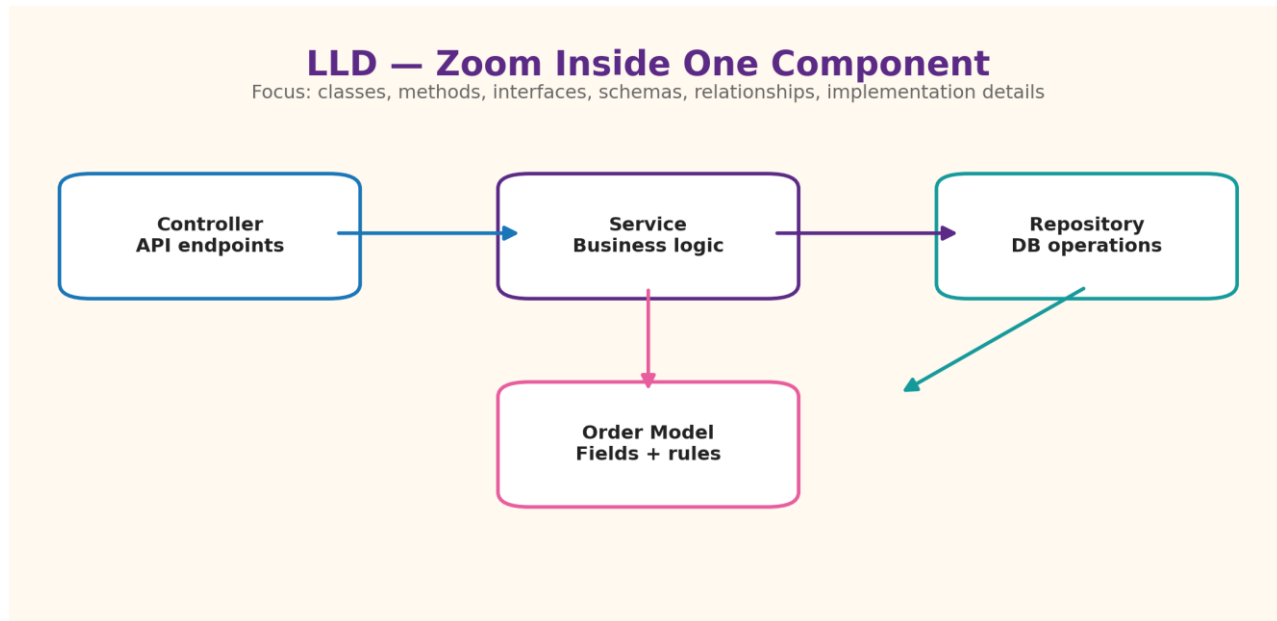


Figure 4 — LLD decomposes one service into implementation-level pieces.

📦 LLD commonly includes

- Classes and objects
- Interfaces
- Methods / functions
- Design patterns
- SOLID principles
- Database entities and detailed schemas
- Validation rules
- Exception handling
- Detailed API request / response models

⚖️ HLD vs LLD

Aspect	HLD	LLD
Scope	Whole system	Individual component
Focus	Architecture	Implementation
Main question	What components and connections?	How does each component work?
Examples	Services, DB, cache, queue, CDN	Classes, methods, interfaces, schemas
Audience	Architects, senior engineers, interviewers	Developers / implementation team



6. Architecture Diagrams

Architecture diagrams turn a complicated system into something you can see. In interviews, a clean diagram is often more useful than a page of text.



A good diagram should show

- Users / clients
- Major services or components
- Databases and storage
- Request / event direction
- Important external systems
- Boundaries between components
- Only the details needed for the current discussion



Common diagram levels

Level	Shows	Example
Context	System + outside actors	Customer → Shopping System → Payment Provider
Container / HLD	Major applications and data stores	Web → API → Services → DB
Component	Internal parts of one service	Controller → Service → Repository
Sequence	Time-ordered interaction	Client → API → DB → Response

Rule: **zoom only when needed.** Start with a simple HLD, then zoom into the component the interviewer asks about.



7. Component Boundaries

A component boundary defines what a component is responsible for and what it should not own. Good boundaries reduce confusion and make systems easier to change.

Component Boundaries — Give Each Part a Clear Responsibility

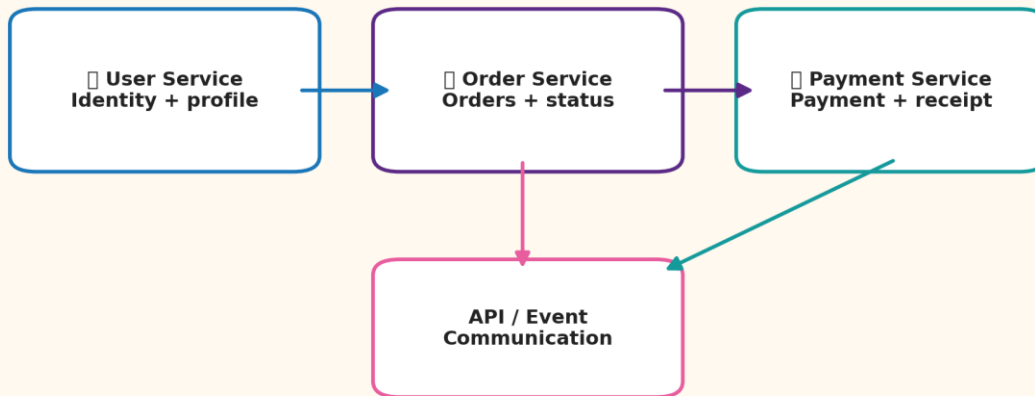


Figure 5 — Each component has a clear responsibility.

? Good boundary questions

- What responsibility belongs here?
- What data does this component own?
- Which APIs or events does it expose?
- Which other components is it allowed to call?
- Can it be changed or scaled independently?
- What happens if this component fails?

💡 Practical rule

A component should have a clear reason to exist. If two components constantly need each other's internal data, your boundary may be wrong.



8. APIs — The Communication Contract

An API defines how one part of a system communicates with another. In a typical web system, clients communicate with backend services using HTTP APIs.

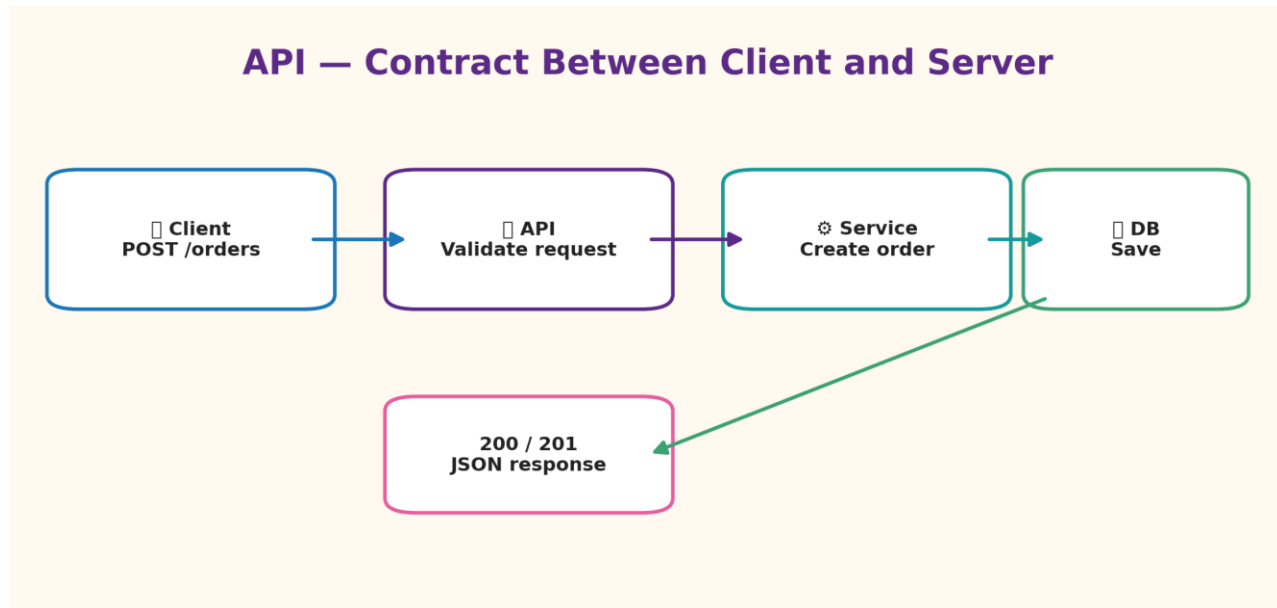


Figure 6 — A simple API request/response flow.



What to define in an API

Part	Example
HTTP method	POST
Endpoint	/orders
Request body	{ productId, quantity }
Authentication	Bearer token / session
Success response	201 Created + order JSON
Error response	400 / 401 / 404 / 409 / 500



Good API design habits

- Use clear resource names.
- Choose HTTP methods consistently.
- Return meaningful status codes.
- Validate input.
- Authenticate and authorize protected operations.
- Design pagination for large collections.
- Think about idempotency for operations such as payments and order creation.
- Keep request/response contracts stable and version carefully when needed.



9. Trade-offs — The Heart of System Design

System Design rarely has one perfect answer. A decision that improves one property can make another property worse.

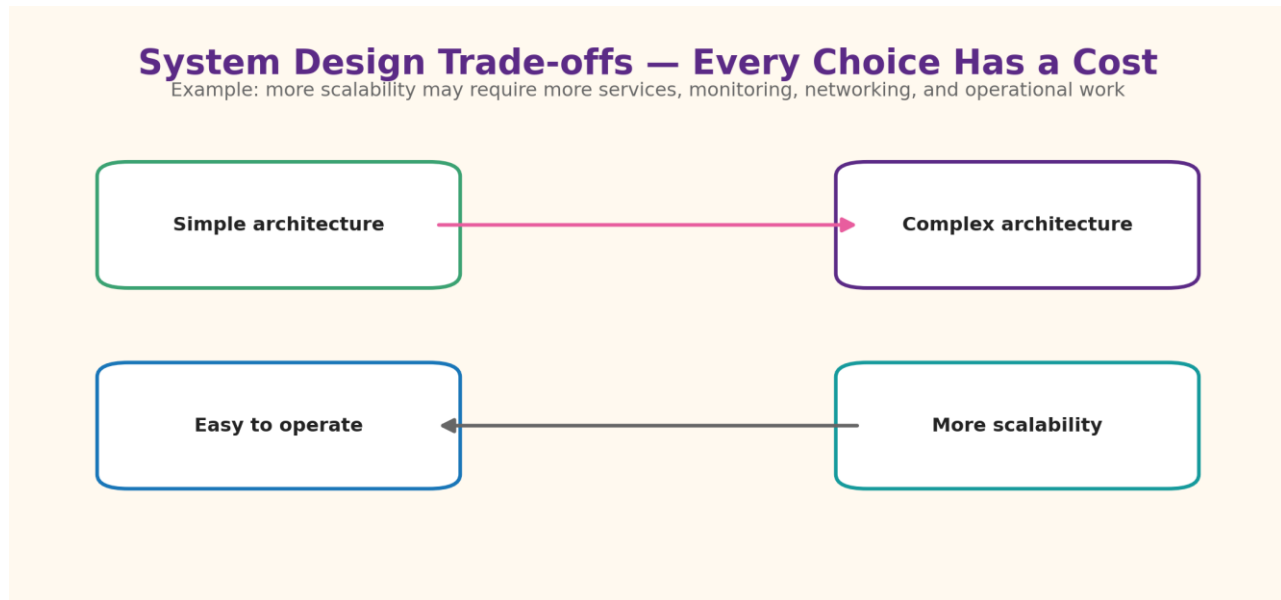


Figure 7 — More capability often comes with more complexity or operational cost.



Common trade-offs

Choice	Benefit	Cost / trade-off
SQL	Strong relationships and transactions	May require more careful scaling at very large scale
NoSQL	Flexible scaling / access patterns	May sacrifice relational features or consistency guarantees
Cache	Lower latency, fewer DB reads	Stale data and invalidation complexity
Horizontal scaling	More capacity and redundancy	Distributed-system complexity
Microservices	Independent deployment/scaling	More networking, observability, and operational complexity
Strong consistency	Fresh, predictable reads	Can increase coordination/latency
Eventual consistency	Better availability/scalability in some designs	Temporary stale data is possible



How to speak about trade-offs in an interview

Use this pattern: **“I choose X because the current requirement is Y. The benefit is A. The cost is B. If the scale or requirement changes to Z, I would reconsider and use X2.”**

10. How All Phase-1 Concepts Fit Together

The concepts are not separate chapters in a real interview. They form one continuous reasoning process.

5. Start with the problem and clarify the scope.
6. Write the functional requirements — the features.
7. Write the non-functional requirements — performance, scale, availability, security, etc.
8. Estimate rough scale so your architecture has realistic constraints.
9. Sketch HLD: clients, services, databases, caches, queues, and external systems.
10. Define component boundaries and responsibilities.
11. Design the important APIs and data flow.
12. Zoom into LLD when a component needs detailed implementation.
13. Explain trade-offs and failure cases.
14. Choose technologies only after the architecture and constraints are clear.

Interview memory

Requirements → HLD → Components → APIs → Data → LLD → Trade-offs. The exact order can vary, but your reasoning should always connect the choices to requirements.

11. One-Page Revision Cheat Sheet

Concept	Remember this
System Design	Design how components work together to satisfy requirements.
Functional	WHAT the system does.
Non-functional	HOW WELL it does it.
HLD	Big picture: components + communication + data flow.
LLD	Detailed implementation: classes + methods + interfaces + schemas.
Architecture diagram	Visual map of components and interactions.
Component boundary	Clear responsibility and ownership.
API	Communication contract between clients/services.
Trade-off	Improving one property may increase cost/complexity elsewhere.

Final takeaway

A strong System Design answer is not a list of technologies. It is a logical story: requirements → constraints → architecture → data flow → scaling/reliability → trade-offs.